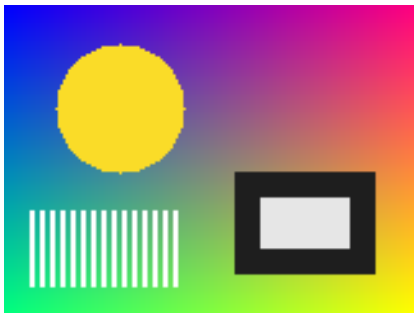
COP 3410 – Assignment 3: Organizing Information and Recursion

Author: Andres Zubizarreta

All code is in `toolkit.py`. `demo.py` runs every function, runs the experiments from 1.6 and saves every input and output image to `examples/`. To use a different image, run `python demo.py photo.jpg cartoon.png`.

The test images were drawn pixel by pixel in `demo.py`: a colorful "photo" with smooth gradients, a sun, a framed box and thin white stripes, and a cartoon house with large flat areas for the paint bucket.

Photo input	Cartoon input
	

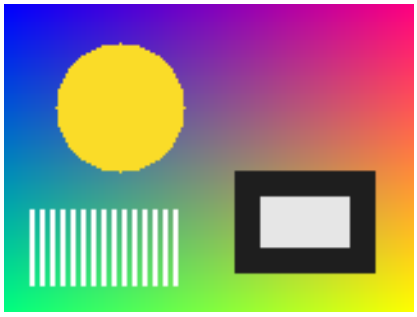
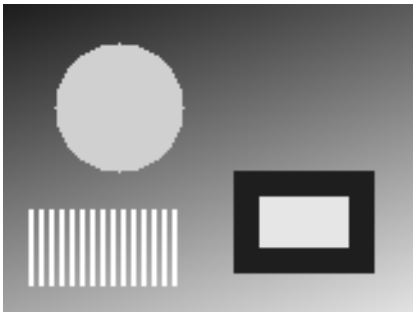
Background question

Why is a pixel a tuple but a row a list? A pixel is always exactly three numbers that belong together, so an immutable, fixed-size tuple fits it and also makes it hashable, which lets us put colors in sets and use them as dictionary keys in Part 2. A row is a sequence of pixels whose length depends on the image and whose entries we need to change (for example `copy[row][col] = new_color` in the paint bucket), so it needs to be a mutable list.

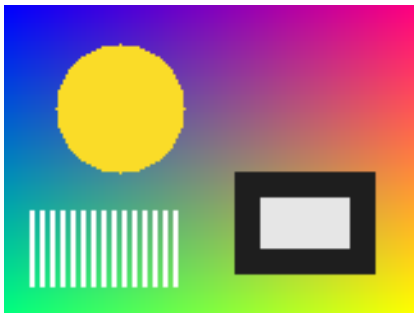
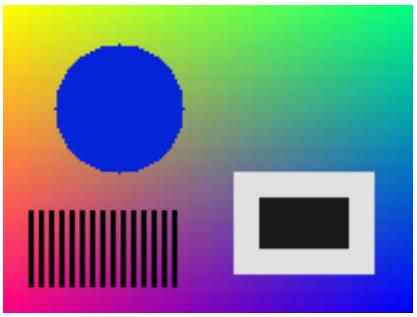
Part 1: Photo filters

1.1 Grayscale

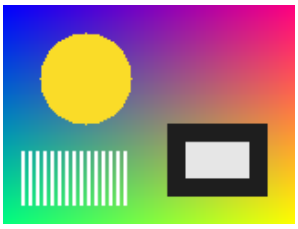
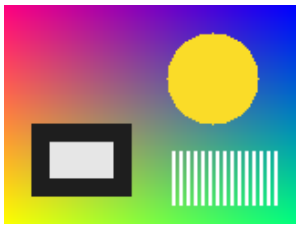
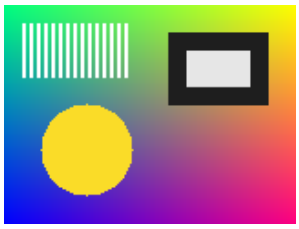
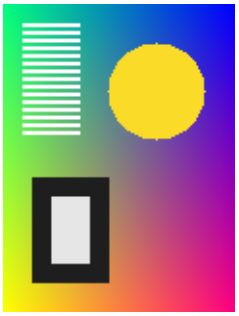
Each pixel becomes `(v, v, v)` with `v = round(0.299R + 0.587G + 0.114B)`.

Input	`grayscale(img)`
	

1.2 Invert

Input	`invert(img)`
	

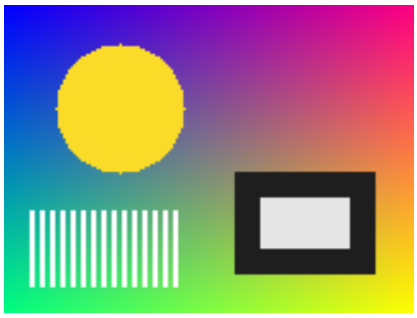
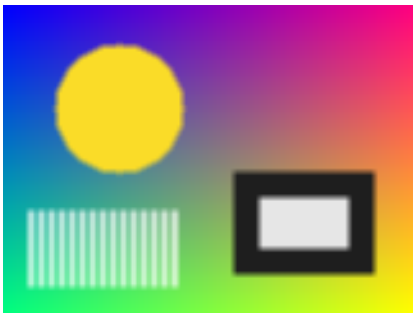
1.3 Flip and rotate

Input	`flip_horizontal`	`flip_vertical`	`rotate_90`
			

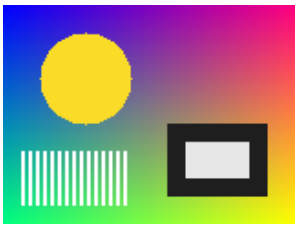
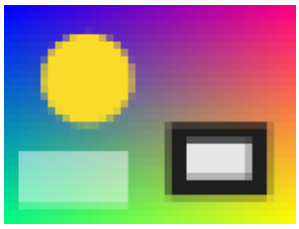
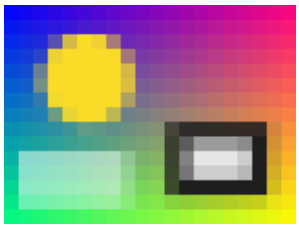
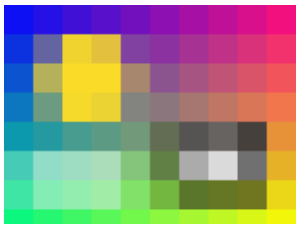
Which flip was clearer? The slicing version of `flip_horizontal` (`row[::-1]`) was clearer, because one short expression says "reverse the row", while the loop in `flip_vertical` makes you track a backwards `range` and its off-by-one end value.

Where does pixel (row, col) go in `rotate_90`? For an image with `height` rows, pixel `(row, col)` moves to `(col, height - 1 - row)`. The first column read from bottom to top becomes the first row, so the output has `width` rows and `height` columns (120 × 160 became 160 × 120).

1.4 Box blur

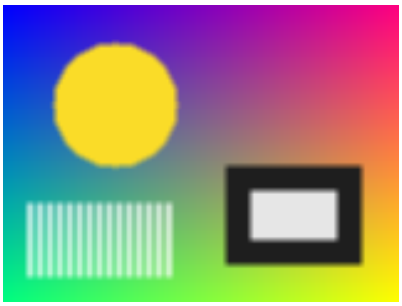
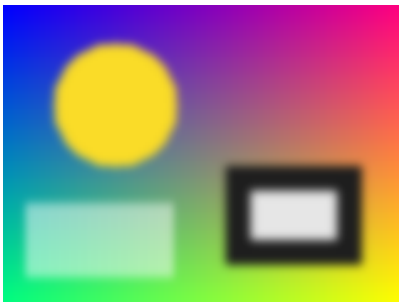
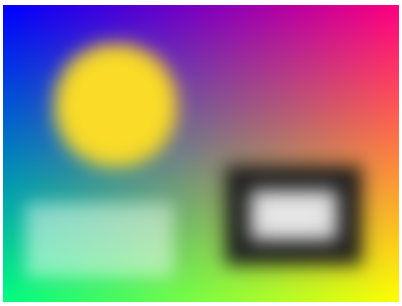
Input	`blur(img)`
	

1.5 Pixelate

Input	`block = 4`	`block = 8`	`block = 16`
			

1.6 Experiments and thinking

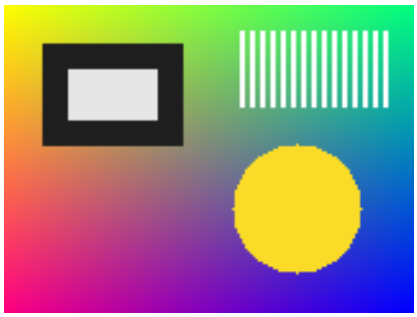
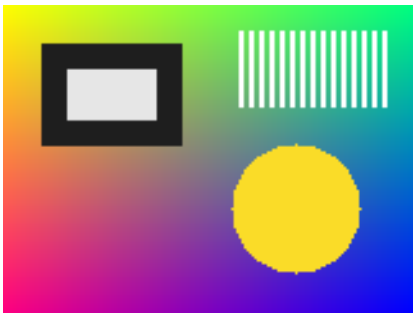
Blur 1, 5 and 20 times.

1 time	5 times	20 times
		

Each pass makes the image softer: the stripes fade and the edges of the sun and the box turn into glows, but the image never becomes one solid color. A blur only averages a pixel with its immediate neighbors, so color spreads just one pixel per pass and pulls nearby pixels toward each other without erasing large-scale differences. After 20 passes the top-left corner was still `(0, 2, 254)` and the bottom-right was still `(254, 252, 4)`. Rounding to whole numbers also stops tiny changes once neighbors are almost equal. The number of distinct colors actually went up (14,143 → 16,769), because sharp edges get replaced by many in-between shades.

Is `invert(invert(img)) == img` always true? Is `grayscale(grayscale(img)) == grayscale(img)`? Both are `True` in code. Inverting twice gives `255 - (255 - x) = x` exactly with integers. Grayscale is stable because a gray pixel `(v, v, v)` gives `(0.299 + 0.587 + 0.114)·v = v`. I also checked every gray level from 0 to 255.

`rotate_90` four times; `flip_horizontal` then `flip_vertical`. Rotating four times is a full 360° turn and gives back the original image (`True`). Flipping horizontally and then vertically sends `(row, col)` to `(height - 1 - row, width - 1 - col)`, which is a 180° rotation, exactly the same as `rotate_90` applied twice (`True`). It is not the same as the original.

`flip_horizontal` then `flip_vertical`	`rotate_90` twice
	

Part 2: Understanding color

2.1 How many colors?

`unique_colors(img)` returned 14,143 distinct colors for the photo. On the tiny 2 × 3 example image with red repeated, it returned only 5 colors because the set keeps each color once. Putting a list in a set fails with `TypeError: cannot use 'list' as a set element (unhashable type: 'list')`, because a list can change, so it cannot have a fixed hash value.

2.2 Brightness histogram

`brightness_histogram(img)` puts each pixel in bin `v * bins // 256`. `print_histogram` scales the bars so the longest one is 50 characters and prints the count at the end:

```
bin  0 |  0
bin  1 | ############################## 1513
bin  2 | ######### 419
bin  3 | #################### 979
bin  4 | ####################### 1109
bin  5 | ############################## 1476
bin  6 | ################################## 1649
bin  7 | ########################################## 2023
bin  8 | ########################################## 1963
bin  9 | ############################ 1307
bin 10 | ####################### 1113
bin 11 | ################# 869
bin 12 | ################ 816
bin 13 | ################################################## 2355
bin 14 | ############## 709
bin 15 | ################## 900
```

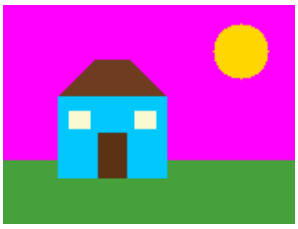
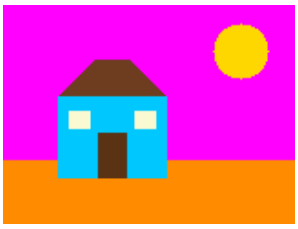
The tall bin 13 comes from the large yellow sun (`v = 208`), the white stripes fill bin 15, and bin 0 is empty because no pixel is darker than `v = 16` (even the dark frame, `(30, 30, 30)`, lands in bin 1).

Part 3: Paint bucket

`flood_fill` follows the given signature and marks its base cases and recursive case in comments. `paint_bucket` copies the image, reads the target color at the starting pixel and starts the fill with an empty `visited` set.

Why a set for `visited`? Checking `(row, col) in visited` is a hash lookup that takes constant time on average with a set, while a list would be scanned from the start on every call, getting slower as more pixels are filled.

Filling the sky, the house and the grass one after another (tolerance 20):

Input	Sky filled	+ house	+ grass
			

The fill stops at the sun, roof, windows and door because their colors are more than 20 away from the target in at least one channel, and the original image was unchanged after each fill. A large region can need one nested call per pixel, more than Python's default limit of about 1000, so `paint_bucket` raises the recursion limit to the number of pixels in the image.

Program output (python demo.py)

```
=== Part 1: Photo filters ===
rotate_90: 120 x 160 (rows x cols) -> 160 x 120
Saved filter outputs to examples

=== 1.6 Experiments ===
blur x 1: 14654 unique colors, top-left (0, 1, 254), bottom-right (254, 254, 2)
blur x 5: 15798 unique colors, top-left (0, 2, 254), bottom-right (254, 252, 3)
blur x20: 16769 unique colors, top-left (0, 2, 254), bottom-right (254, 252, 4)
original: 14143 unique colors
invert(invert(img)) == img: True
grayscale(grayscale(img)) == grayscale(img): True
  ...and for every gray level 0-255: True
rotate_90 x4 == img: True
flip_horizontal then flip_vertical == rotate_90 x2: True
flip_horizontal then flip_vertical == img: False

=== Part 2: Color ===
unique colors: 14143
unique_colors(tiny 2x3 image): [(0, 0, 0), (0, 255, 0), (128, 128, 128), (255, 0, 0), (255, 255, 255)]
Putting a list in a set raises: TypeError("cannot use 'list' as a set element (unhashable type: 'list')")
brightness_histogram: {0: 0, 1: 1513, 2: 419, 3: 979, 4: 1109, 5: 1476, 6: 1649, 7: 2023, 8: 1963, 9: 1307, 10: 1113, 11
bin  0 |  0
bin  1 | ################################## 1513
bin  2 | ######### 419
bin  3 | ###################### 979
bin  4 | ######################### 1109
bin  5 | ################################## 1476
bin  6 | ##################################### 1649
bin  7 | ############################################## 2023
bin  8 | ########################################### 1963
bin  9 | ############################ 1307
bin 10 | ######################### 1113
bin 11 | ################## 869
bin 12 | ################# 816
bin 13 | ################################################## 2355
bin 14 | ############### 709
bin 15 | ################### 900

=== Part 3: Paint bucket ===
original unchanged: True
sky pixel after fill: (255, 0, 255) | sun pixel untouched: (255, 215, 0)
```

Code: toolkit.py

```
"""Image-processing toolkit for COP 3410 Assignment 3.

An image is a list of rows, each row is a list of pixels, and each pixel is a
tuple (r, g, b). Always index as pixels[row][col], which is pixels[y][x].
Every filter returns a NEW image and never changes the input in place.
"""

import sys


# ---------------------------------------------------------------------------
# Part 1: Photo filters
# ---------------------------------------------------------------------------

def gray_value(pixel):
    """Return the weighted grayscale value v of one (r, g, b) pixel."""
    r, g, b = pixel
    return round(0.299 * r + 0.587 * g + 0.114 * b)


def grayscale(pixels):
    """1.1: Return a new image where every pixel is (v, v, v)."""
    result = []
    for row in pixels:
        new_row = []
        for pixel in row:
            v = gray_value(pixel)
            new_row.append((v, v, v))
        result.append(new_row)
    return result


def invert(pixels):
    """1.2: Return the photo negative: each pixel becomes (255-r, 255-g, 255-b)."""
    return [[(255 - r, 255 - g, 255 - b) for (r, g, b) in row] for row in pixels]


def flip_horizontal(pixels):
    """1.3: Mirror the image left to right, using list slicing."""
    return [row[::-1] for row in pixels]


def flip_vertical(pixels):
    """1.3: Mirror the image top to bottom, using a loop."""
    result = []
    for r in range(len(pixels) - 1, -1, -1):
        result.append(list(pixels[r]))
    return result


def rotate_90(pixels):
    """1.3: Rotate the image 90 degrees clockwise.

    The output has `width` rows and `height` columns. Pixel (row, col) of the
    input moves to (col, height - 1 - row) of the output, so output pixel
    (r, c) comes from input pixel (height - 1 - c, r).
    """
    height = len(pixels)
    width = len(pixels[0])
    result = []
    for r in range(width):
        new_row = []
        for c in range(height):
            new_row.append(pixels[height - 1 - c][r])
        result.append(new_row)
    return result


def average_color(colors):
    """Return the rounded average (r, g, b) of a non-empty list of pixels."""
```

```
    n = len(colors)
    total_r = total_g = total_b = 0
    for r, g, b in colors:
        total_r += r
        total_g += g
        total_b += b
    return (round(total_r / n), round(total_g / n), round(total_b / n))


def blur(pixels):
    """1.4: 3 x 3 box blur.

    Each new pixel is the average of itself and its up-to-8 neighbors. Edge
    and corner pixels only average the neighbors that exist.
    """
    height = len(pixels)
    width = len(pixels[0])
    result = []
    for r in range(height):
        new_row = []
        for c in range(width):
            box = []
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < height and 0 <= nc < width:
                        box.append(pixels[nr][nc])
            new_row.append(average_color(box))
        result.append(new_row)
    return result


def pixelate(pixels, block):
    """1.5: Split the image into block x block squares and paint each square
    with its average color. Squares on the right/bottom edge may be smaller."""
    height = len(pixels)
    width = len(pixels[0])
    result = [list(row) for row in pixels]
    for top in range(0, height, block):
        for left in range(0, width, block):
            bottom = min(top + block, height)
            right = min(left + block, width)
            square = [pixels[r][c] for r in range(top, bottom) for c in range(left, right)]
            color = average_color(square)
            for r in range(top, bottom):
                for c in range(left, right):
                    result[r][c] = color
    return result


# ---------------------------------------------------------------------------
# Part 2: Understanding color
# ---------------------------------------------------------------------------

def unique_colors(pixels):
    """2.1: Return a set of all distinct colors in the image."""
    colors = set()
    for row in pixels:
        for pixel in row:
            colors.add(pixel)
    return colors


def brightness_histogram(pixels, bins=16):
    """2.2: Return a dictionary mapping each bin number to a pixel count.

    A pixel with grayscale value v goes in bin v * bins // 256. Every bin from
    0 to bins - 1 is present, even if its count is 0.
    """
    hist = {b: 0 for b in range(bins)}
    for row in pixels:
        for pixel in row:
            v = gray_value(pixel)
```

```
            hist[v * bins // 256] += 1
    return hist


def print_histogram(hist, max_width=50):
    """2.2: Draw the histogram as a text bar chart.

    Bars are scaled so the tallest bin is max_width characters wide, because a
    real photo has thousands of pixels per bin.
    """
    largest = max(hist.values()) if hist else 0
    for b in sorted(hist):
        count = hist[b]
        length = round(count * max_width / largest) if largest else 0
        print(f"bin {b:2d} | {'#' * length} {count}")


# ---------------------------------------------------------------------------
# Part 3: Paint bucket (recursive flood fill)
# ---------------------------------------------------------------------------

def within_tolerance(color, target, tolerance):
    """Two colors are similar if each channel differs by at most tolerance."""
    for a, b in zip(color, target):
        if abs(a - b) > tolerance:
            return False
    return True


def flood_fill(pixels, row, col, target, new_color, tolerance, visited):
    """Recolor every pixel connected to (row, col) whose color is within
    `tolerance` of `target`. Moves are up, down, left and right only.

    `visited` is a set of (row, col) tuples. A set is a better choice than a
    list because checking `(row, col) in visited` takes constant time on
    average with a set (it is a hash lookup), but a list has to be scanned
    from the start every time, which gets slower as more pixels are filled.
    """
    # Base cases: off the image? already visited? color too different?
    if row < 0 or row >= len(pixels) or col < 0 or col >= len(pixels[0]):
        return
    if (row, col) in visited:
        return
    if not within_tolerance(pixels[row][col], target, tolerance):
        return

    # Recursive case: recolor this pixel, mark it visited,
    #                 then call flood_fill on the 4 neighbors.
    # Each call marks one more pixel as visited, so the number of pixels
    # left to explore keeps shrinking until every branch hits a base case.
    pixels[row][col] = new_color
    visited.add((row, col))
    flood_fill(pixels, row - 1, col, target, new_color, tolerance, visited)  # up
    flood_fill(pixels, row + 1, col, target, new_color, tolerance, visited)  # down
    flood_fill(pixels, row, col - 1, target, new_color, tolerance, visited)  # left
    flood_fill(pixels, row, col + 1, target, new_color, tolerance, visited)  # right


def paint_bucket(pixels, row, col, new_color, tolerance=30):
    """Return a copy of the image with the area connected to (row, col) filled
    with new_color. The original image is not modified."""
    copy = [list(r) for r in pixels]
    target = pixels[row][col]
    # A large region can need one nested call per pixel, which is more than
    # Python's default limit of about 1000.
    needed = len(pixels) * len(pixels[0]) + 1000
    if sys.getrecursionlimit() < needed:
        sys.setrecursionlimit(needed)
    flood_fill(copy, row, col, target, new_color, tolerance, set())
    return copy
```

Code: demo.py

```
"""Run every function in toolkit.py on sample images and run the experiments
from section 1.6.

Usage:
    python demo.py                      # uses the generated sample images
    python demo.py photo.jpg            # uses your own photo for Parts 1 and 2
    python demo.py photo.jpg cartoon.png

Input and output images are written to the examples/ folder.
"""

import os
import sys

from helpers import load_image, save_image, blank_image
from toolkit import (
    grayscale, invert, flip_horizontal, flip_vertical, rotate_90, blur,
    pixelate, unique_colors, brightness_histogram, print_histogram,
    paint_bucket,
)

OUT = "examples"


# ---------------------------------------------------------------------------
# Sample images, drawn pixel by pixel (no drawing library needed)
# ---------------------------------------------------------------------------

def fill_rect(pixels, top, left, bottom, right, color):
    for r in range(max(top, 0), min(bottom, len(pixels))):
        for c in range(max(left, 0), min(right, len(pixels[0]))):
            pixels[r][c] = color


def fill_circle(pixels, cy, cx, radius, color):
    for r in range(len(pixels)):
        for c in range(len(pixels[0])):
            if (r - cy) ** 2 + (c - cx) ** 2 <= radius ** 2:
                pixels[r][c] = color


def make_photo(width=160, height=120):
    """A colorful image with smooth gradients, shapes and fine detail."""
    pixels = []
    for r in range(height):
        row = []
        for c in range(width):
            red = 255 * c // (width - 1)
            green = 255 * r // (height - 1)
            blue = 255 - (red + green) // 2
            row.append((red, green, blue))
        pixels.append(row)
    fill_circle(pixels, 40, 45, 25, (250, 220, 40))
    fill_rect(pixels, 65, 90, 105, 145, (30, 30, 30))
    fill_rect(pixels, 75, 100, 95, 135, (230, 230, 230))
    # Thin stripes show off blur and pixelate
    for c in range(10, 70, 4):
        fill_rect(pixels, 80, c, 110, c + 2, (255, 255, 255))
    return pixels


def make_cartoon(width=160, height=120):
    """A cartoon scene with large flat areas for the paint bucket."""
    pixels = blank_image(width, height, (120, 190, 240))       # sky
    fill_circle(pixels, 25, 130, 15, (255, 215, 0))            # sun
    fill_rect(pixels, 85, 0, height, width, (70, 160, 60))     # grass
    fill_rect(pixels, 50, 30, 95, 90, (200, 60, 50))           # house
    for i in range(20):                                        # roof
        fill_rect(pixels, 30 + i, 60 - i - 10, 31 + i, 60 + i + 10, (110, 60, 30))
    fill_rect(pixels, 70, 52, 95, 68, (90, 50, 20))            # door
```

```
    fill_rect(pixels, 58, 36, 68, 48, (250, 250, 210))         # window
    fill_rect(pixels, 58, 72, 68, 84, (250, 250, 210))         # window
    return pixels


# ---------------------------------------------------------------------------
# Demo
# ---------------------------------------------------------------------------

def path(name):
    return os.path.join(OUT, name)


def part1(img):
    print("=== Part 1: Photo filters ===")
    save_image(grayscale(img), path("1_1_grayscale.png"))
    save_image(invert(img), path("1_2_invert.png"))
    save_image(flip_horizontal(img), path("1_3_flip_horizontal.png"))
    save_image(flip_vertical(img), path("1_3_flip_vertical.png"))
    rotated = rotate_90(img)
    save_image(rotated, path("1_3_rotate_90.png"))
    print(f"rotate_90: {len(img)} x {len(img[0])} (rows x cols) -> "
          f"{len(rotated)} x {len(rotated[0])}")
    save_image(blur(img), path("1_4_blur.png"))
    for block in (4, 8, 16):
        save_image(pixelate(img, block), path(f"1_5_pixelate_{block}.png"))
    print("Saved filter outputs to", OUT)


def experiments(img):
    print("\n=== 1.6 Experiments ===")

    # Blur 1, 5 and 20 times
    blurred = img
    for times in range(1, 21):
        blurred = blur(blurred)
        if times in (1, 5, 20):
            save_image(blurred, path(f"1_6_blur_x{times}.png"))
            colors = unique_colors(blurred)
            print(f"blur x{times:2d}: {len(colors):5d} unique colors, "
                  f"top-left {blurred[0][0]}, bottom-right {blurred[-1][-1]}")
    print(f"original: {len(unique_colors(img)):5d} unique colors")

    # Involution / idempotence
    print("invert(invert(img)) == img:", invert(invert(img)) == img)
    gray = grayscale(img)
    print("grayscale(grayscale(img)) == grayscale(img):", grayscale(gray) == gray)
    every_gray = [[(v, v, v) for v in range(256)]]
    print("  ...and for every gray level 0-255:", grayscale(every_gray) == every_gray)

    # Rotations and flips
    four = rotate_90(rotate_90(rotate_90(rotate_90(img))))
    print("rotate_90 x4 == img:", four == img)
    both_flips = flip_vertical(flip_horizontal(img))
    two_rotations = rotate_90(rotate_90(img))
    save_image(both_flips, path("1_6_flip_h_then_v.png"))
    save_image(two_rotations, path("1_6_rotate_90_x2.png"))
    print("flip_horizontal then flip_vertical == rotate_90 x2:",
          both_flips == two_rotations)
    print("flip_horizontal then flip_vertical == img:", both_flips == img)


def part2(img):
    print("\n=== Part 2: Color ===")
    print("unique colors:", len(unique_colors(img)))
    tiny = [[(255, 0, 0), (0, 255, 0), (255, 0, 0)],
            [(0, 0, 0), (128, 128, 128), (255, 255, 255)]]
    print("unique_colors(tiny 2x3 image):", sorted(unique_colors(tiny)))
    try:
        {[255, 0, 0]}
    except TypeError as error:
        print("Putting a list in a set raises:", repr(error))
    hist = brightness_histogram(img)
```

```
    print("brightness_histogram:", hist)
    print_histogram(hist)


def part3(cartoon):
    print("\n=== Part 3: Paint bucket ===")
    sky = paint_bucket(cartoon, 5, 5, (255, 0, 255), tolerance=20)
    save_image(sky, path("3_paint_bucket_sky.png"))
    house = paint_bucket(sky, 80, 35, (0, 200, 255), tolerance=20)
    save_image(house, path("3_paint_bucket_house.png"))
    grass = paint_bucket(house, 110, 150, (255, 140, 0), tolerance=20)
    save_image(grass, path("3_paint_bucket_grass.png"))
    print("original unchanged:", cartoon[5][5] == (120, 190, 240))
    print("sky pixel after fill:", sky[5][5], "| sun pixel untouched:", sky[25][130])


def main():
    os.makedirs(OUT, exist_ok=True)
    if len(sys.argv) > 1:
        img = load_image(sys.argv[1])
    else:
        img = make_photo()
    if len(sys.argv) > 2:
        cartoon = load_image(sys.argv[2], max_size=120)
    else:
        cartoon = make_cartoon()
    save_image(img, path("0_input_photo.png"))
    save_image(cartoon, path("0_input_cartoon.png"))

    part1(img)
    experiments(img)
    part2(img)
    part3(cartoon)


if __name__ == "__main__":
    main()
```

Code: helpers.py (provided)

```
from PIL import Image

def load_image(path, max_size=200):
    """Open an image and return it as a list of rows of (r, g, b) tuples.
    The image is shrunk so its longest side is at most max_size pixels."""
    img = Image.open(path).convert("RGB")
    img.thumbnail((max_size, max_size))
    w, h = img.size
    data = list(img.getdata())
    return [data[r * w:(r + 1) * w] for r in range(h)]

def load_square(path, size=128):
    """Open an image cropped and resized to size x size (use a power of 2)."""
    img = Image.open(path).convert("RGB")
    side = min(img.size)
    img = img.crop((0, 0, side, side)).resize((size, size))
    data = list(img.getdata())
    return [data[r * size:(r + 1) * size] for r in range(size)]

def save_image(pixels, path):
    """Save a list of rows of (r, g, b) tuples as an image file."""
    h, w = len(pixels), len(pixels[0])
    img = Image.new("RGB", (w, h))
    img.putdata([tuple(p) for row in pixels for p in row])
    img.save(path)

def blank_image(width, height, color=(255, 255, 255)):
    """Return a new image filled with one color."""
    return [[color for _ in range(width)] for _ in range(height)]
```